

Section 94 Editorial — Half-Planes And Circle Geometry

A. Half-Plane Region

How to find it

Sort directed boundaries by angle, discard weaker parallel copies, and maintain feasible consecutive intersections in a deque.

Correctness

The deque invariant retains exactly the processed region boundaries in angular order; every removed corner violates the new constraint and can never return. Closing yields precisely the intersection polygon.

Complexity

$O(n \log n)$ time and $O(n)$ memory.

C++

```
#include <bits/stdc++.h>
using namespace std;
const double EPS=1e-10,PI=acos(-1.0);
struct P {
    double x,y;
    P operator+(P b)const {
        return {
            x+b.x,y+b.y
        }
    };
    P operator-(P b)const {
        return {
            x-b.x,y-b.y
        }
    };
    P operator*(double k)const {
        return {
            x*k,y*k
        }
    };
};
double cross(P a,P b) {
    return a.x*b.y-a.y*b.x;
}
double dot(P a,P b) {
    return a.x*b.x+a.y*b.y;
}
double norm(P a) {
    return hypot(a.x,a.y);
```

```

}
struct L {
    P p,v;
    double angle;
}
;
P meet(L a,L b) {
    double t=cross(b.p-a.p,b.v)/cross(a.v,b.v);
    return a.p+a.v*t;
}
vector<P>halfplanes(vector<pair<P,P>>raw) {
    vector<L>a;
    for(auto [p,q]:raw) {
        P v=q-p;
        a.push_back({p,v,atan2(v.y,v.x)});
    }
    sort(a.begin(),a.end(),[](L x,L y){return x.angle<y.angle;});
    vector<L>u;
    for(L z:a) {
        if(!u.empty()&&abs(cross(u.back().v,z.v))<=EPS&&dot(u.back().v,z.v)>0) {
            if(cross(u.back().v,z.p-u.back().p)>EPS)u.back()=z;
        }
        else u.push_back(z);
    }
    deque<L>q;
    deque<P>pts;
    auto inside=[](L l,P p) {
        return cross(l.v,p-l.p)>=-EPS;
    }
    ;
    for(L z:u) {
        while(!pts.empty()&&!inside(z,pts.back()))pts.pop_back(),q.pop_back();
        while(!pts.empty()&&!inside(z,pts.front()))pts.pop_front(),q.pop_front();
        if(!q.empty()&&abs(cross(q.back().v,z.v))<=EPS)return {
        }
        ;
        if(!q.empty())pts.push_back(meet(q.back(),z));
        q.push_back(z);
    }
    while(!pts.empty()&&!inside(q.front(),pts.back()))pts.pop_back(),q.pop_back();
    while(!pts.empty()&&!inside(q.back(),pts.front()))pts.pop_front(),q.pop_front();
    if(q.size()<3)return {
    }
    ;
    pts.push_back(meet(q.back(),q.front()));
    return vector<P>(pts.begin(),pts.end());
}
double area(vector<P>p) {
    double s=0;
    for(int i=0;i<(int)p.size();i++)s+=cross(p[i],p[(i+1)%p.size()]);
    return abs(s)/2;
}
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cout<<fixed<<setprecision(10);
    int n;
    cin>>n;
    vector<pair<P,P>>l(n);
    for(auto&[a,b]:l)cin>>a.x>>a.y>>b.x>>b.y;
    cout<<area(halfplanes(l))<<'\\n';
}

```

Python

```
import sys, math, random
from collections import deque
EPS = 1e-10

def add(a, b):
    return (a[0] + b[0], a[1] + b[1])

def sub(a, b):
    return (a[0] - b[0], a[1] - b[1])

def mul(a, k):
    return (a[0] * k, a[1] * k)

def cross(a, b):
    return a[0] * b[1] - a[1] * b[0]

def dot(a, b):
    return a[0] * b[0] + a[1] * b[1]

def intersection(a, b):
    t = cross(sub(b[0], a[0]), b[1]) / cross(a[1], b[1])
    return add(a[0], mul(a[1], t))

def halfplanes(raw):
    lines = []
    for a, b in raw:
        v = sub(b, a)
        lines.append((a, v, math.atan2(v[1], v[0])))
    lines.sort(key=lambda z: z[2])
    unique = []
    for p, v, angle in lines:
        if unique and abs(cross(unique[-1][1], v)) <= EPS and (dot(unique[-1][1], v) > 0):
            if cross(unique[-1][1], sub(p, unique[-1][0])) > EPS:
                unique[-1] = (p, v, angle)
            else:
                unique.append((p, v, angle))
    dq = deque()
    points = deque()

    def inside(line, p):
        return cross(line[1], sub(p, line[0])) >= -EPS
    for line in unique:
        while points and (not inside(line, points[-1])):
            points.pop()
            dq.pop()
        while points and (not inside(line, points[0])):
            points.popleft()
            dq.popleft()
        if dq and abs(cross(dq[-1][1], line[1])) <= EPS:
            return []
        if dq:
            points.append(intersection(dq[-1], line))
            dq.append(line)
        while points and (not inside(dq[0], points[-1])):
            points.pop()
            dq.pop()
        while points and (not inside(dq[-1], points[0])):
            points.popleft()
            dq.popleft()
    if len(dq) < 3:
        return []
```

```

    points.append(intersection(dq[-1], dq[0]))
    return list(points)

def area(poly):
    return abs(sum((cross(poly[i], poly[(i + 1) % len(poly)])) for i in range(len(poly))))) / 2 if len(poly) >= 3 else 0.0

def main():
    d = list(map(float, sys.stdin.buffer.read().split()))
    n = int(d[0])
    raw = []
    for i in range(n):
        x1, y1, x2, y2 = d[1 + 4 * i:5 + 4 * i]
        raw.append((x1, y1), (x2, y2))
    print(f'{area(halfplanes(raw)):.10f}')
if __name__ == '__main__':
    main()

```

B. Circle Overlap

How to find it

Separate disjoint and containment cases before using the two-sector lens formula.

Correctness

In the proper-overlap case the common region partitions into two sectors minus their center triangles; the other classifications are immediate.

Complexity

$O(1)$ time and memory.

C++

```

#include <bits/stdc++.h>
using namespace std;
const double EPS=1e-10,PI=acos(-1.0);
struct P {
    double x,y;
    P operator+(P b)const {
        return {
            x+b.x,y+b.y
        }
    };
}
P operator-(P b)const {
    return {
        x-b.x,y-b.y
    };
}
P operator*(double k)const {
    return {
        x*k,y*k
    };
}
}
;

```

```

double cross(P a,P b) {
    return a.x*b.y-a.y*b.x;
}
double dot(P a,P b) {
    return a.x*b.x+a.y*b.y;
}
double norm(P a) {
    return hypot(a.x,a.y);
}
struct L {
    P p,v;
    double angle;
}
;
P meet(L a,L b) {
    double t=cross(b.p-a.p,b.v)/cross(a.v,b.v);
    return a.p+a.v*t;
}
struct C {
    P p;
    double r;
}
;
double overlap(C a,C b) {
    double d=norm(a.p-b.p);
    if(d>=a.r+b.r)return 0;
    if(d<=abs(a.r-b.r))return PI*min(a.r,b.r)*min(a.r,b.r);
    double x=acos(clamp((d*d+a.r*a.r-b.r*b.r)/(2*d*a.r),-1.0,1.0)),y=acos(clamp((d*d+b.r*b.r-a.r*a.r)/(2*d*b.r),-1.0,1.0));
    return a.r*a.r*x+b.r*b.r*y-d*a.r*sin(x);
}
int tangents(C a,C b) {
    double d=norm(a.p-b.p);
    if(d<=EPS)return abs(a.r-b.r)<=EPS?-1:0;
    int ans=0;
    for(double z:{abs(a.r-b.r),a.r+b.r})ans+=d>z+EPS?2:(abs(d-z)<=EPS);
    return ans;
}
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cout<<fixed<<setprecision(10);
    C a,b;
    cin>>a.p.x>>a.p.y>>a.r>>b.p.x>>b.p.y>>b.r;
    cout<<overlap(a,b)<<'\n';
}

```

Python

```

import sys, math, random
from collections import deque
EPS = 1e-10

def overlap(c1, c2):
    x1, y1, r1 = c1
    x2, y2, r2 = c2
    d = math.hypot(x1 - x2, y1 - y2)
    if d >= r1 + r2:
        return 0.0
    if d <= abs(r1 - r2):
        return math.pi * min(r1, r2) ** 2
    a = math.acos(max(-1, min(1, (d * d + r1 * r1 - r2 * r2) / (2 * d * r1))))
    b = math.acos(max(-1, min(1, (d * d + r2 * r2 - r1 * r1) / (2 * d * r2))))

```

```

    return r1 * r1 * a + r2 * r2 * b - d * r1 * math.sin(a)

def main():
    d = list(map(float, sys.stdin.buffer.read().split()))
    print(f'{overlap(d[:3], d[3:6]):.10f}')
if __name__ == '__main__':
    main()

```

C. Common Tangent Count

How to find it

Count external and internal tangent families by comparing center distance with radius difference and sum.

Correctness

Each positive perpendicular offset produces two mirror lines, zero produces their merged tangent, and a negative squared offset produces none.

Complexity

$O(1)$.

C++

```

#include <bits/stdc++.h>
using namespace std;
const double EPS=1e-10,PI=acos(-1.0);
struct P {
    double x,y;
    P operator+(P b)const {
        return {
            x+b.x,y+b.y
        }
    };
}
P operator-(P b)const {
    return {
        x-b.x,y-b.y
    };
}
P operator*(double k)const {
    return {
        x*k,y*k
    };
}
;
double cross(P a,P b) {
    return a.x*b.y-a.y*b.x;
}
double dot(P a,P b) {
    return a.x*b.x+a.y*b.y;
}
double norm(P a) {
    return hypot(a.x,a.y);
}

```

```

struct L {
    P p,v;
    double angle;
}
;
P meet(L a,L b) {
    double t=cross(b.p-a.p,b.v)/cross(a.v,b.v);
    return a.p+a.v*t;
}
struct C {
    P p;
    double r;
}
;
double overlap(C a,C b) {
    double d=norm(a.p-b.p);
    if(d>=a.r+b.r)return 0;
    if(d<=abs(a.r-b.r))return PI*min(a.r,b.r)*min(a.r,b.r);
    double x=acos(clamp((d*d+a.r*a.r-b.r*b.r)/(2*d*a.r),-1.0,1.0)),y=acos(clamp((d*d+b.r*b.r-a.r*a.r)/(2*d*b.r),-1.0,1.0));
    return a.r*a.r*x+b.r*b.r*y-d*a.r*sin(x);
}
int tangents(C a,C b) {
    double d=norm(a.p-b.p);
    if(d<=EPS)return abs(a.r-b.r)<=EPS?-1:0;
    int ans=0;
    for(double z:{abs(a.r-b.r),a.r+b.r})ans+=d>z+EPS?2:(abs(d-z)<=EPS);
    return ans;
}
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cout<<fixed<<setprecision(10);
    C a,b;
    cin>>a.p.x>>a.p.y>>a.r>>b.p.x>>b.p.y>>b.r;
    cout<<tangents(a,b)<<'\\n';
}

```

Python

```

import sys, math, random
from collections import deque
EPS = 1e-10

def tangent_count(c1, c2):
    x1, y1, r1 = c1
    x2, y2, r2 = c2
    d = math.hypot(x1 - x2, y1 - y2)
    if d <= EPS:
        return -1 if abs(r1 - r2) <= EPS else 0
    ans = 0
    for z in (abs(r1 - r2), r1 + r2):
        ans += 2 if d > z + EPS else 1 if abs(d - z) <= EPS else 0
    return ans

def main():
    d = list(map(float, sys.stdin.buffer.read().split()))
    print(tangent_count(d[:3], d[3:6]))
if __name__ == '__main__':
    main()

```

D. Largest Inscribed Circle

How to find it

Offset every counterclockwise edge inward by radius r ; test whether the shifted half-planes intersect, then binary-search r .

Correctness

A center supports radius r exactly when its distance from every edge is at least r , which is exactly membership in all shifted half-planes. Feasibility is monotone.

Complexity

$O(n \log n \log \text{precision})$ with general HPI.

C++

```
#include <bits/stdc++.h>
using namespace std;
const double EPS=1e-10,PI=acos(-1.0);
struct P {
    double x,y;
    P operator+(P b)const {
        return {
            x+b.x,y+b.y
        }
    };
}
P operator-(P b)const {
    return {
        x-b.x,y-b.y
    };
}
P operator*(double k)const {
    return {
        x*k,y*k
    };
}
}
;
double cross(P a,P b) {
    return a.x*b.y-a.y*b.x;
}
double dot(P a,P b) {
    return a.x*b.x+a.y*b.y;
}
double norm(P a) {
    return hypot(a.x,a.y);
}
struct L {
    P p,v;
    double angle;
}
;
P meet(L a,L b) {
    double t=cross(b.p-a.p,b.v)/cross(a.v,b.v);
    return a.p+a.v*t;
}
```

```

vector<P>halfplanes(vector<pair<P,P>>raw) {
    vector<L>a;
    for(auto [p,q]:raw) {
        P v=q-p;
        a.push_back({p,v,atan2(v.y,v.x)});
    }
    sort(a.begin(),a.end(),[](L x,L y){return x.angle<y.angle;});
    vector<L>u;
    for(L z:a) {
        if(!u.empty()&&abs(cross(u.back().v,z.v))<=EPS&&dot(u.back().v,z.v)>0) {
            if(cross(u.back().v,z.p-u.back().p)>EPS)u.back().v=z;
        }
        else u.push_back(z);
    }
    deque<L>q;
    deque<P>pts;
    auto inside=[](L l,P p) {
        return cross(l.v,p-l.p)>=-EPS;
    };
    for(L z:u) {
        while(!pts.empty()&&!inside(z,pts.back()))pts.pop_back(),q.pop_back();
        while(!pts.empty()&&!inside(z,pts.front()))pts.pop_front(),q.pop_front();
        if(!q.empty()&&abs(cross(q.back().v,z.v))<=EPS)return {
        };
        if(!q.empty())pts.push_back(meet(q.back(),z));
        q.push_back(z);
    }
    while(!pts.empty()&&!inside(q.front(),pts.back()))pts.pop_back(),q.pop_back();
    while(!pts.empty()&&!inside(q.back(),pts.front()))pts.pop_front(),q.pop_front();
    if(q.size()<3)return {
    };
    pts.push_back(meet(q.back(),q.front()));
    return vector<P>(pts.begin(),pts.end());
}

double area(vector<P>p) {
    double s=0;
    for(int i=0;i<(int)p.size();i++)s+=cross(p[i],p[(i+1)%p.size()]);
    return abs(s)/2;
}

vector<P>shifted(vector<P>p,double r) {
    vector<pair<P,P>>l;
    for(int i=0;i<(int)p.size();i++) {
        P q=p[(i+1)%p.size()],v=q-p[i],s= {
            -v.y*r/norm(v),v.x*r/norm(v)
        };
        l.push_back({p[i]+s,q+s});
    }
    return halfplanes(l);
}

double inradius(vector<P>p) {
    double lo=0,hi=1;
    for(P x:p)for(P y:p)hi=max(hi,norm(x-y));
    for(int z=0;z<70;z++) {
        double m=(lo+hi)/2;
        (shifted(p,m).empty()?hi:lo)=m;
    }
    return lo;
}

int main() {

```

```

ios::sync_with_stdio(false);
cin.tie(nullptr);
cout<<fixed<<setprecision(10);
int n;
cin>>n;
vector<P>p(n);
for(auto&x:p)cin>>x.x>>x.y;
cout<<inradius(p)<<'\n';
}

```

Python

```

import sys, math, random
from collections import deque
EPS = 1e-10

def add(a, b):
    return (a[0] + b[0], a[1] + b[1])

def sub(a, b):
    return (a[0] - b[0], a[1] - b[1])

def mul(a, k):
    return (a[0] * k, a[1] * k)

def cross(a, b):
    return a[0] * b[1] - a[1] * b[0]

def dot(a, b):
    return a[0] * b[0] + a[1] * b[1]

def norm(a):
    return math.hypot(a[0], a[1])

def intersection(a, b):
    t = cross(sub(b[0], a[0]), b[1]) / cross(a[1], b[1])
    return add(a[0], mul(a[1], t))

def halfplanes(raw):
    lines = []
    for a, b in raw:
        v = sub(b, a)
        lines.append((a, v, math.atan2(v[1], v[0])))
    lines.sort(key=lambda z: z[2])
    unique = []
    for p, v, angle in lines:
        if unique and abs(cross(unique[-1][1], v)) <= EPS and (dot(unique[-1][1], v) > 0):
            if cross(unique[-1][1], sub(p, unique[-1][0])) > EPS:
                unique[-1] = (p, v, angle)
            else:
                unique.append((p, v, angle))
    dq = deque()
    points = deque()

    def inside(line, p):
        return cross(line[1], sub(p, line[0])) >= -EPS
    for line in unique:
        while points and (not inside(line, points[-1])):
            points.pop()
            dq.pop()
        while points and (not inside(line, points[0])):
            points.popleft()

```

```

        dq.popleft()
    if dq and abs(cross(dq[-1][1], line[1])) <= EPS:
        return []
    if dq:
        points.append(intersection(dq[-1], line))
    dq.append(line)
    while points and (not inside(dq[0], points[-1])):
        points.pop()
        dq.pop()
    while points and (not inside(dq[-1], points[0])):
        points.popleft()
        dq.popleft()
    if len(dq) < 3:
        return []
    points.append(intersection(dq[-1], dq[0]))
    return list(points)

def shifted_polygon(poly, r):
    lines = []
    for i, p in enumerate(poly):
        q = poly[(i + 1) % len(poly)]
        v = sub(q, p)
        length = norm(v)
        shift = (-v[1] * r / length, v[0] * r / length)
        lines.append((add(p, shift), add(q, shift)))
    return halfplanes(lines)

def inradius(poly):
    low = 0.0
    high = max(max((x for x, y in poly)) - min((x for x, y in poly)), max((y for x, y in poly)) - min((y for x, y in poly)))
    for _ in range(70):
        mid = (low + high) / 2
        if shifted_polygon(poly, mid):
            low = mid
        else:
            high = mid
    return low

def main():
    d = list(map(float, sys.stdin.buffer.read().split()))
    n = int(d[0])
    p = [tuple(d[1 + 2 * i:3 + 2 * i]) for i in range(n)]
    print(f'{inradius(p):.10f}')
if __name__ == '__main__':
    main()

```

Predicate and degeneracy audit

Half-plane intersection keeps the left side of every directed boundary. Equal-direction lines require retaining the stricter constraint; opposite parallel lines may create a strip or emptiness. Deque removal is justified because a corner rejected by the new half-plane can never return after later directions are added.

Circle formulas are used only after classifying disjointness, containment, tangency, and coincident centers. For the inscribed-circle search, inward offset feasibility is monotone. Test translation and scaling, nearly parallel lines, zero-area intersections, and tangent or almost-tangent disks. Clamp only mathematically bounded trigonometric inputs, not arbitrary predicates.